Предобученный энкодер

От восстановления слов к классификации обращений

Какие знания мы переносим

Слайд 1. Предобученный энкодер

На второй лекции классификатор учился по словам нашего небольшого корпуса. На пятой мы задавали матрицы attention вручную и прослеживали вычисления. Теперь возьмём энкодер, параметры которого уже получены при обучении на других текстах. Он преобразует обращение в контекстные векторы, а мы научимся использовать их для знакомой задачи выбора отдела.

Слайд 2. Архитектура, веса и наша задача

Здесь полезно различать архитектуру, веса и прикладную систему. Архитектура описывает, какие вычисления выполняются. Веса задают конкретные обученные преобразования. Прикладная система определяет, как подать текст, какой выход взять и как связать его с шестью категориями университета «Север».

Предобучение может помочь связать «учебник» и «книгу» или различать употребления «ключа». Но категория «библиотека» в нашем классификаторе должна быть связана с примерами нашей разметки. Если организация поменяет порядок обработки обращений, языковые закономерности могут остаться прежними, а прикладные метки потребуется пересмотреть.

Для опыта выбран русскоязычный энкодер rubert-tiny2. В используемой конфигурации три слоя, двенадцать голов attention и скрытая размерность 312. Эти числа описывают конкретную модель. Её карточка рекомендует получать представление текста из первой специальной позиции CLS и нормировать полученный вектор.

Сначала разберём идею предобучения BERT, затем проследим путь одного пакета через настоящий токенизатор и энкодер. rubert-tiny2 относится к этому семейству, но история его обучения отличается от рецепта исходной статьи: автор отдельно описывает приближение представлений предложений к LaBSE и настройку на отношениях между текстами.

BERT восстанавливает выбранные токены

Слайд 3. Восстановление видит обе стороны

В предложении «Студент положил книгу в сумку, чтобы нести её домой» скроем «сумку». Получится задание с пропуском внутри текста. Левое окружение говорит о помещении книги во что-то, правое — о переноске. Энкодер может использовать обе стороны, чтобы восстановить выбранный токен.

Это masked language modeling, MLM. В авторегрессионной языковой модели известен только префикс, а следующий токен ещё не написан. Здесь исходный текст уже дан, но часть входа изменена. Правая часть допустима для задачи восстановления и была бы недоступным будущим в причинном предсказании.

Слайд 4. Выбранные цели не все скрыты

В исходном BERT выбирали 15% токенов как цели. Из выбранных 80% заменяли на [MASK], 10% — случайным токеном, 10% оставляли без изменения. Эти доли относятся к разным множествам. Для условных 1000 токенов выбирается около 150 целей, из них примерно 120 скрываются, 15 заменяются и 15 остаются прежними. Это иллюстрация ожидаемых пропорций, а не точные количества в каждом маленьком пакете.

При вычислении потери используется вероятность исходного токена в каждой выбранной позиции. Как и раньше, низкая вероятность правильной цели даёт большой отрицательный логарифм. Параметры энкодера и выходного слоя изменяются, чтобы восстановление по контексту стало точнее. Неизменённая часть выбранных целей также входит в эту задачу; нельзя отождествить множество целей только с видимыми [MASK].

В исходной работе использовалась и next sentence prediction: модель различала специально сформированные пары сегментов. В последующих моделях набор задач менялся. Поэтому знакомая архитектура и наличие разделителей ещё не означают, что любой checkpoint прошёл одинаковое предобучение.

Слайд 5. Три разные роли маски

Слово «маска» здесь легко перепутать с пятой лекцией. [MASK] — токен, которым заменили содержимое. Padding-маска указывает, какие ключи относятся к техническому дополнению. Causal-маска запрещает будущие позиции. У них разные роли, хотя в разговоре используется одно слово.

При классификации обычного обращения нам не нужно специально скрывать его слова. Предобучение уже изменило параметры через задачу восстановления. Теперь подадим полный доступный текст и извлечём его представления.

Токенизатор задаёт настоящий вход

Слайд 6. Токены принадлежат этой модели

На первой лекции один адрес мог стать пятью токенами при выбранном регулярном выражении. У предобученного энкодера также есть собственные правила и словарь. Их нужно использовать вместе с моделью: идентификатор токена указывает на конкретную строку таблицы эмбеддингов.

Возьмём «Как продлить книгу?» и «Спасибо». Токенизатор добавит специальные позиции, разобьёт обычные слова и вернёт идентификаторы. Вопрос «Как продлить книгу?» даёт шесть позиций: [CLS], Как, продлить, книгу, ?, [SEP]. «Спасибо» занимает три позиции, включая CLS и SEP. Мы посмотрим и числа, и восстановленные названия токенов. Чужие номера из примера другого BERT здесь не подходят: одинаковое число в другом словаре может означать другой элемент.

Слайд 7. Padding выравнивает пакет

Чтобы подать несколько текстов одним пакетом, строки дополняются до общей длины. `input_ids` содержит идентификаторы, `attention_mask` — указание содержательных и технических позиций в ожидаемом моделью формате. Если в пакете B текстов длины L после дополнения, обе таблицы имеют форму B×L. Для двух выбранных строк B=2 и L=6: у вопроса шесть единиц в маске, у «Спасибо» — три единицы и три нуля.

Для этой модели первая позиция должна быть CLS. Она тоже занимает место в длине входа, как и SEP. Если отключить добавление специальных токенов и по-прежнему взять позицию с индексом ноль, получится вектор первого обычного токена. Форма массива будет выглядеть правильной, но заявленный способ представления изменится.

В лабораторной используется предел 128 токенов, включая специальные позиции. Сначала посчитаем полные длины без усечения, затем число строк, которые превысят этот предел. У модели в конфигурации поддерживается больше позиций; предел конкретного опыта задаётся отдельно и должен быть известен при разборе результата.

Padding и усечение действуют противоположным образом. Первый добавляет технические позиции короткому тексту, второе удаляет часть длинного. Маска помогает исключить заполнители из участия в нужных операциях, но не восстанавливает отрезанную просьбу. Этот случай отдельно проверим в конце работы.

Из последовательности нужен один вектор

Слайд 8. Вектор для каждой позиции

Энкодер возвращает `last_hidden_state` формы B×L×312. Для каждого текста и каждой позиции есть вектор из 312 координат. Для классификатора обращений нужен один вектор на текст, поэтому требуется выбрать способ объединения позиций — pooling.

Слайд 9. CLS выбирается у всех текстов

В нашем опыте используется нормированный CLS, как рекомендует автор модели. Запись `hidden[:, 0, :]` выбирает первую позицию у всех B текстов и даёт B×312. После L2-нормировки каждая ненулевая строка имеет единичную длину. Нормируем по оси признаков, чтобы соседние тексты пакета не участвовали в знаменателе.

Слайд 10. Padding попадает в среднее

У других моделей может быть рекомендовано усреднение токенов. Рассмотрим его отдельно на маленьком примере: H=((1;0), (0;1), (9;9)), маска m=(1;1;0). Первые две позиции содержательные, третья обозначает padding.

Слайд 11. Маска выбирает слагаемые

Обычное среднее всех трёх равно (10/3;10/3), а среднее выбранных маской — (0,5;0,5).

Формула masked mean — $\Sigma_t m_t h_t$ / $\Sigma_t m_t$. В числителе закрытая позиция умножается на ноль, в знаменателе учитываются только выбранные позиции. Если их нет, требуется отдельное правило вместо деления на ноль. В реальном энкодере специальные токены тоже могут присутствовать; наш пример выделяет только влияние padding.

Почему вектор padding вообще может быть ненулевым? Маска запрещает использовать соответствующий ключ как источник содержательной информации, но не обязана занулять весь выход этой позиции. На выход влияют остаточные связи, позиционные и другие преобразования. Поэтому нельзя сначала усреднить всё, а затем считать, что сама attention-маска уже исправила среднее.

На практике проверим короткий текст отдельно и рядом с более длинным. Соседние заполнители не должны существенно менять его представление при корректных масках и режиме применения. Малую разницу округлений допускаем; заметное расхождение потребует проверить подготовку входа и pooling.

Какие параметры будут обучаться

Слайд 12. Что меняется при дообучении

Самый прямой вариант переноса — оставить энкодер фиксированным и обучить новую модель на его выходах. Мы получим векторы train и подадим их логистической регрессии вместе с шестью метками. Меняться будут коэффициенты классификатора, а преобразование текста внутри энкодера останется прежним.

При полном fine-tuning обучаются и параметры самого энкодера. Его представления могут приспособиться к нашей задаче. Тогда потребуется выбирать режим оптимизации и следить за переносом на отложенные данные: хорошее согласие с небольшим train ещё не показывает, что новая настройка полезна для новых обращений.

Можно также обучать ограниченную часть параметров или добавленные модули. Адаптеры и низкоранговая адаптация (LoRA) уменьшают число изменяемых параметров по сравнению с полным дообучением. Это третий вариант организации обучения. В обязательном опыте будем использовать первый, чтобы исследовать готовое представление отдельно от его дообучения.

Слайд 13. Три настройки решают разное

В коде встречаются три похожих на первый взгляд действия. `eval()` переключает поведение модулей, например отключает случайное выбрасывание признаков в dropout. `requires_grad_(False)` задаёт, что для соответствующих параметров не нужен градиент. `inference_mode()` выполняет проход без обычного построения графа autograd.

Они отвечают на разные вопросы. Энкодер в `eval()` ещё может участвовать в обучении, если градиенты и оптимизатор разрешены. Замороженная модель в режиме обучения всё ещё может выдавать разные результаты из-за dropout. Для извлечения фиксированных признаков зададим нужные условия явно.

Новая классификационная голова не входит автоматически в исходный checkpoint. Если добавить случайно инициализированный слой с шестью выходами, он технически вернёт шесть оценок, но ещё не научится нашим категориям. Именно размеченный train связывает готовые признаки с назначением отделов.

Проследим один пакет в коде

Слайд 14. От пакета к признакам

Рассмотрим один проход от списка строк к признакам. Токенизатор и энкодер уже загружены вместе. Переменная `texts` содержит пакет обращений.

```
encoder.eval()
encoder.requires_grad_(False)

with torch.inference_mode():
    batch = tokenizer(texts, padding=True, truncation=True,
                      max_length=128, return_tensors="pt")
    hidden = encoder(**batch).last_hidden_state
    vectors = torch.nn.functional.normalize(hidden[:, 0, :], dim=1)
    features = vectors.cpu().numpy()
```

Подпишем формы при B=2. `batch` содержит таблицы 2×L. `hidden` имеет форму 2×L×312. После выбора первой позиции остаётся 2×312, и эта форма сохраняется после нормировки и перехода в NumPy.

Слайд 15. Индекс меняет форму результата

Если случайно написать `hidden[0, 0, :]`, пропадёт весь второй текст: получится только 312 чисел первого.

Для коллекции нужен цикл по пакетам с объединением строк результата. Порядок должен совпадать с исходными текстами и их метками. Даже идеальные векторы не помогут классификатору, если затем отсортировать их отдельно от меток и тем самым связать с чужими категориями.

Проверим число строк, конечность значений и нормы. Затем сравним одинаковое обращение при двух запусках и при другом составе пакета. Каждая проверка относится к своему свойству: правильная форма не обнаружит неверный выбор первой позиции, а единичная норма не докажет сохранение соответствия меткам.

После извлечения признаков возвращаемся к знакомому scikit-learn: `fit` логистической регрессии на train, `predict` на validation и test. Применение фиксированного энкодера к отдельному тестовому тексту отличается от обучения словаря и IDF по всему корпусу. В первом случае заранее заданное преобразование вычисляет признаки; во втором статистика теста меняет само преобразование.

Для NER сохраняются позиции

Слайд 16. NER: границы и типы

Теперь изменим форму результата. Вместо одной темы сообщения нужно выделить имена и организации. Эта задача называется named entity recognition, NER. Один документный вектор уже не даёт прямого соответствия границам слов, поэтому здесь используются представления позиций и сведения об их связи с исходным текстом.

Возьмём «Анна Петрова учится в университете Север». По нашей инструкции имя включает «Анна Петрова», организация — «университете Север». В BIO-разметке B

отмечает начало сущности, I — продолжение того же типа, O — позиции вне сущностей. Для шести слов получаем B-PER, I-PER, O, O, B-ORG, I-ORG.

Если другая инструкция выделяет в организации только «Север», границы будут другими. Прежде чем сравнивать модели, нужно договориться об этом с разметчиками. Тот же вопрос уже возникал с неоднозначными категориями обращений: эталон выражает выбранную постановку.

Слайд 17. Граница имени тоже часть ответа

Допустим, эталон — «Анна Петрова», а система вернула только «Петрова» с типом PER. При строгой оценке границы не совпали. Получается один лишний предсказанный фрагмент и один пропущенный эталон. Высокая доля правильно размеченных O в остальном тексте не исправляет эту ошибку выделения.

Слайд 18. От слова к подтокенам

Слово может разбиться на несколько токенов модели. Тогда метки исходных слов нельзя просто прикрепить к выходам по порядку: после первого разбиения соответствие сдвинется. Один из подходов — обучать метку только на первом подтокене слова, а остальные исключать из потери. Другой распределяет метки по всем частям с согласованным правилом BIO.

Реальное разбиение посмотрим у токенизатора. Если одно слово имени разбивается на две части, обе относятся к одному слову исходного текста. Нам нужны сведения о принадлежности токенов словам или символьные смещения. При извлечении результата они позволяют вернуться к точному фрагменту текста.

В лабораторной BIO будет ручным упражнением. Обученная голова решает классификацию обращений, а не NER. Для полноценного извлечения понадобятся отдельные примеры с границами и типами, соответствующее обучение и оценка сущностей.

Что изменилось относительно TF-IDF

Слайд 19. Качество на тесте выросло

Сравним две заранее заданные системы. Первая — char_wb TF-IDF с фрагментами 3–5 и логистической регрессией. Вторая — нормированный CLS rubert-tiny2 и такая же регрессия с C=1. Разделение данных, шесть меток и функции оценки одинаковы. Энкодер фиксирован, новые коэффициенты классификатора обучаются только по train.

В выполненном опыте на test символьный вариант получил accuracy около 0,7917 и macro-F1 0,7652, вариант с энкодером — 0,9000 и 0,8863. Это улучшение на

конкретном корпусе. Разберём, какие обращения за ним стоят, и проверим показатели отдельных категорий.

Слайд 20. Recall поступления — 0,40

В показанных вариантах система на основе TF-IDF ошиблась в классификации обращений о монографии и подборе источников для курсовой, а система с энкодером определила их категории правильно. Однако у класса поступления recall равен только 0,40: двенадцать из двадцати его строк ушли в другие категории. Общая accuracy 0,90 этого не показывает.

Вопрос об изменении направления в заявлении абитуриента энкодер направил в учебный процесс, тогда как TF-IDF справился. Для объяснения полезно прочитать исходную просьбу, посмотреть соседние примеры и границу категорий. Тематическая близость сама по себе не гарантирует нужного организационного маршрута.

Эти числа не используются для дальнейшего подбора C или pooling на test. Мы сравниваем фиксированные системы и разбираем наблюдения. Для следующего изменения понадобятся отдельное решение о настройке и сохранённая граница между разработкой и проверкой.

Как исчезает важная часть обращения

Слайд 21. Короткие тексты не усечены

В основном корпусе полные длины со специальными токенами лежат между 8 и 21. При пределе 128 ни одно обращение не усечено. Поэтому объяснять ошибки основной классификации потерянным концом здесь нельзя.

Слайд 22. Усечение убрало просьбу

Построим отдельный диагностический текст: восемьдесят раз повторим строку `"Уточняю ранее отправленный вопрос. "`, а в конце добавим `"Как продлить книгу?"`. Точка и пробел входят в повторяемую строку. Полная длина получилась 566 токенов. После усечения осталось 128, и просьба о книге исчезла. Декодируем сохранённый вход, чтобы увидеть это непосредственно.

В таком случае классификатор получает другое содержание, чем читатель полного письма. Улучшение выходной головы не вернёт недоступную просьбу. Для длинных документов стоит рассмотреть разбиение на части, выбор существенных фрагментов или другой способ учёта длинного контекста, а затем проверить его на соответствующих примерах.

Слайд 23. Найдём причину ошибки

Есть и другой недостаток входа: сообщение само по себе неоднозначно. Вопрос «Можно ли отправить пакет документов по почте?» отнесён к категории поступления в задуманной автором ситуации, но сама фраза может относиться к разным процедурам. В показанном варианте с приветствием энкодер выбрал библиотеку. Это расхождение с меткой одновременно поднимает вопрос о достаточности контекста.

При разборе ошибки различайте три случая: нужные сведения были отрезаны, сведений не было в самом сообщении, сведения были доступны, но модель их не использовала как нужно. Для каждого требуется своё изменение системы или данных.

Готовые признаки стали частью системы

Слайд 24. От токенов к прикладному ответу

Теперь путь обращения выглядит так: текст проходит через связанный с моделью токенизатор, энкодер возвращает представления позиций, выбранный pooling даёт один вектор, обученная голова выбирает отдел. Мы можем указать, какие параметры взяты из предобучения и какие получены на нашем train.

Проверим понимание на двух изменениях кода. Что произойдёт, если оставить `eval()`, но разрешить градиенты и обновления оптимизатора? А если заменить `hidden[:, 0, :]` на `hidden[0, 0, :]`? Первый случай касается обучаемости параметров, второй — потери оси пакета. Одинаково успешное выполнение Python ещё не означает, что решается прежняя задача.

На следующем занятии используем генеративную модель. Она будет выдавать свободный текст и принимать инструкцию. Вопросы о формате входа, обученной задаче и оценке прикладного результата останутся такими же конкретными: что модель увидела, что должна вернуть и как мы проверим её ответ.

Для дальнейшего чтения

– Devlin et al. BERT, NAACL 2019 — MLM и перенос на прикладные задачи.

– Карточка cointegrated/rubert-tiny2 — особенности модели и нормированный CLS.

– Transformers: padding и truncation — подготовка пакетов.

– PyTorch: inference_mode — режим вычисления представлений.

Конфигурация и ревизия модели сохранены в `course/models-lock.json`. Примеры обращений, ручное среднее и BIO-разметка созданы для курса.